



Examen

INDICACIONES

- Este examen se realiza de forma individual.
- El examen debe ser rendido en la plataforma <http://codetrack.cl/>
- Para entrar debe escribir su correo institucional y como password su RUT (solo números).
- Lea cada pregunta con atención y programe en la plataforma.
- No se permite el uso de dispositivos electrónicos como celulares, relojes inteligentes, tablets, etc. Quien sea sorprendido/a con alguno de ellos, se le evaluará con la nota mínima.
- Cualquier consulta sobre la plataforma debe ser resuelta con los profesores a cargo.
- Puede usar su cheat sheet impreso y entregado en clases.

Problemas

1. Clasificación de mediciones de brillo con funciones y bucles.

Enunciado

Durante una noche de observación se registran valores de flujo relativo de una estrella. Para resumir las mediciones se usará una función, un ciclo y condicionales.

Use la siguiente lista:

$$\text{flujos} = [0.82, 1.05, 1.18, 0.76, 1.34, 0.91, 1.27, 0.69].$$

Una medición se clasifica según su flujo F :

$$F < 0.85 \implies \text{baja}, \quad 0.85 \leq F \leq 1.15 \implies \text{normal}, \quad F > 1.15 \implies \text{alta}.$$

Realice las siguientes operaciones:

- Defina la lista `flujos`.
- Cree una función llamada `clasificar_flujo(F)` que reciba un flujo y retorne el texto "baja", "normal" o "alta" según los criterios anteriores.
- Recorra la lista con un ciclo `for`. Para cada flujo, use la función creada, imprima el índice, el flujo y su clasificación.
- Cuente cuántas mediciones quedan en cada categoría. Use contadores simples que aumenten dentro del ciclo.
- Calcule el promedio de los flujos usando una suma acumulada y `len(flujos)`.
- Use un condicional final para indicar si la noche fue estable o no: si la cantidad de mediciones normales es mayor o igual que la suma de mediciones bajas y altas, imprima "noche estable"; en caso contrario, imprima "noche variable".

2. Modelo de decaimiento radiactivo con NumPy y Matplotlib.

Enunciado

El decaimiento radiactivo de una muestra puede modelarse mediante la ley exponencial

$$N(t) = N_0 e^{-\lambda t},$$

donde $N(t)$ representa el número de núcleos radiactivos presentes en el instante t , N_0 corresponde al número inicial de núcleos y λ es la constante de decaimiento radiactivo.

Use los valores

$$N_0 = 1000, \quad \lambda = 0,23.$$

Realice las siguientes operaciones:

- Importe `numpy` y `matplotlib.pyplot`. Cree un arreglo `t` con 300 valores igualmente espaciados entre 0 y 20 usando `np.linspace`.
- Cree una función llamada `decaimiento(t, N0, lam)` que retorne $N(t)$.
- Evalúe la función para obtener el arreglo `N`.
- Calcule el número mínimo de núcleos radiactivos, el número máximo y el número promedio del arreglo `N`.
- Determine el tiempo aproximado en que $N(t)$ es menor que 250 por primera vez. Puede usar `np.where`.
- Grafique $N(t)$ en función del tiempo usando `plt.plot`. Agregue una línea horizontal en $N = 250$, etiquetas, título, leyenda y grilla.

3. Limpieza y resumen de observaciones astronómicas con pandas.

Enunciado

Se tiene un conjunto pequeño de observaciones astronómicas. Cada observación tiene un nombre, un tipo de objeto, un flujo medido, una distancia y una estimación del ruido instrumental.

Use el siguiente bloque para crear el DataFrame:

```
1 datos = {  
2     "nombre": ["A1", "A2", "B1", "B2", "C1", "C2", "D1", "D2"],  
3     "tipo": ["estrella", "estrella", "galaxia", "galaxia",  
4             "cuáasar", "cuáasar", "estrella", "galaxia"],  
5     "flujo": [12.5, 8.1, 20.4, None, 5.2, 6.3, 10.9, 18.7],  
6     "distancia_pc": [4.2, 7.8, 120.0, 98.0, 300.0, None, 5.5, 110.0],  
7     "ruido": [0.8, 0.9, 2.5, 2.1, 0.6, 0.7, None, 2.2],  
8 }
```

Realice las siguientes operaciones:

- Importe pandas, numpy y matplotlib.pyplot. Cree un DataFrame llamado obs a partir del diccionario datos.
- Inspeccione el DataFrame mostrando las primeras filas, dimensiones, tipos de datos y cantidad de valores faltantes por columna.
- Cree un DataFrame llamado obs_limpias eliminando las filas con datos faltantes en flujo, distancia_pc o ruido. Use copy().
- En obs_limpias, cree una columna llamada snr usando

$$\text{snr} = \frac{\text{flujo}}{\text{ruido}}.$$

- Agrupe obs_limpias por tipo. Calcule la cantidad de observaciones por tipo usando la columna tipo.
- Cree una copia sin la columna nombre. Agrupe esa copia por tipo y calcule el promedio y la desviación estándar por tipo. Use numeric_only=True si lo necesita.
- Filtre las observaciones con $\text{snr} > 10$ y $\text{distancia_pc} < 150$. Muestre solamente nombre, tipo, flujo, distancia_pc y snr.
- Haga un gráfico de dispersión simple con distancia_pc en el eje horizontal y snr en el eje vertical. Agregue etiquetas, título y grilla.